# EE115B *Digital Circuits*

Instructor: Chenxi Xiao

Part of slides are from
© Pearson Education
Prof. Hengzhao Yang
Prof. Zhifeng Zhu
Prof. Yajun Ha


上海科技大学
ShanghaiTech University

# Number Systems & Conversions

Summary: How to perform those conversions?

| | |
|---|---|
| 2->8 | 16->2 |
| 2->10 | 16->8 |
| 2->16 | 16->10 |
| 8->2 | 10->2 |
| 8->10 | 10->8 |
| 8->16 | 10->16 |

# 2's Complement Form

In the two's complement representation, a negative number is represented by:
the bit pattern corresponding to the bitwise NOT (i.e. the "complement") of the positive number plus one, i.e. to the ones' complement plus one.

$$N^* = 2^n - N$$

Can represent 256 numbers!
-128 to +127

**Eight-bit two's complement**

| Binary value | Two's complement interpretation | Unsigned interpretation |
|---|---|---|
| 00000000 | 0 | 0 |
| 00000001 | 1 | 1 |
| ⋮ | ⋮ | ⋮ |
| 01111110 | 126 | 126 |
| 01111111 | 127 | 127 |
| 10000000 | −128 | 128 |
| 10000001 | −127 | 129 |
| 10000010 | −126 | 130 |
| ⋮ | ⋮ | ⋮ |
| 11111110 | −2 | 254 |
| 11111111 | −1 | 255 |



https://en.wikipedia.org/wiki/Signed_number_representations

# 2's Complement Form: Example of Subtraction

- Subtracting +3 (subtrahend) from +8 (minuend) is equivalent to adding −3 to +8.
- The sign of a positive or negative binary number is changed by taking its 2's complement. (proof it)

| | | |
|---|---|---|
| | 00001000 | Minuend (+8) |
| | + 11111101 | 2's complement of subtrahend (−3) |
| Discard carry ⟶ | **1 00000101** | Difference (+5) |

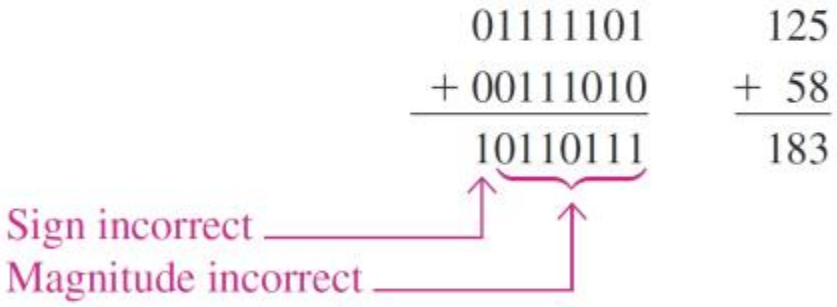
- It's important to determine the number of bits in advance

| | | | | | |
|---|---|---|---|---|---|
| +13 | 0 | 01101 | +13 | 0 | 01101 |
| +10 | 0 | 01010 | −10 | 1 | 10110 |
| +23 | 0 | 10111 | +3 | 0 | 00011 |
| −13 | 1 | 10011 | −13 | 1 | 10011 |
| +10 | 0 | 01010 | −10 | 1 | 10110 |
| −3 | 1 | 11101 | −23 | 1 | 01001 |

| | | | | | |
|---|---|---|---|---|---|
| +13 | 0 | 1101 | +13 | 0 | 1101 |
| +10 | 0 | 1010 | −10 | 1 | 0110 |
| +23 | 1 | 0111 | +3 | 0 | 0011 |
| −13 | 1 | 0011 | −13 | 1 | 0011 |
| +10 | 0 | 1010 | −10 | 1 | 0110 |
| −3 | 1 | 1101 | −23 | 0 | 1001 |

# Overflow Condition

- When two numbers are added and the number of bits required to represent the sum exceeds the number of bits in the two numbers, an overflow results as indicated by an incorrect sign bit. An overflow can occur only when both numbers are positive or both numbers are negative. If the sign bit of the result is different than the sign bit of the numbers that are added, overflow is indicated.

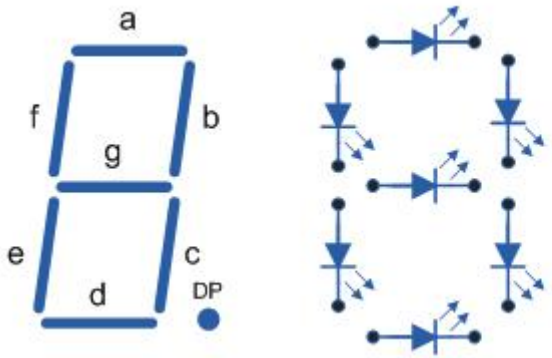



Think about it:

- How to confidently perform the calculation like the last page?

# Binary Coded Decimal: BCD

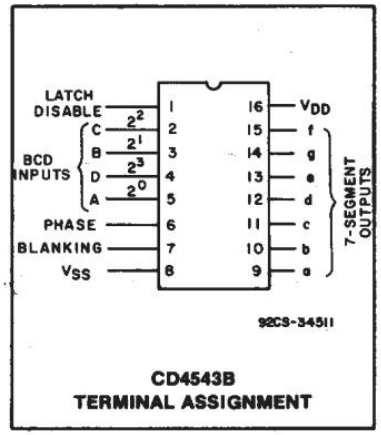
Directly using binary code for many systems is not straightforward (such as display systems).

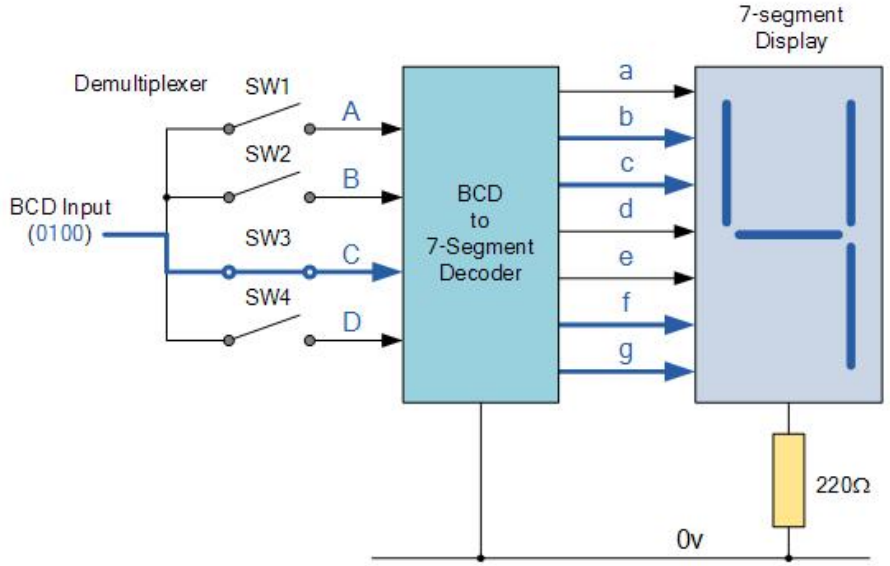



## Display Decoder

A Display Decoder is a combinational circuit which decodes and n-bit input value into a number of output lines to drive a display

**CD4543B Types**




https://www.electronics-tutorials.ws/combination/comb_6.html
https://www.ti.com/lit/ds/symlink/cd4543b.pdf

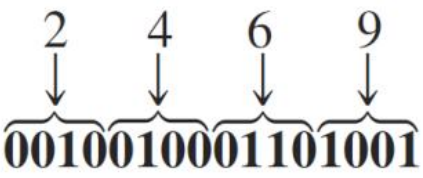
# Binary Coded Decimal: BCD

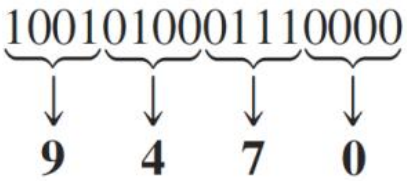
- BCD means that each decimal digit, 0 through 9, is represented by a binary code of four bits.
- The 8421 code is a type of BCD code. The designation 8421 indicates the binary weights of the four bits ($2^3$, $2^2$, $2^1$, $2^0$).

Decimal/BCD conversion.

| Decimal Digit | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| BCD | 0000 | 0001 | 0010 | 0011 | 0100 | 0101 | 0110 | 0111 | 1000 | 1001 |

- 1010, 1011, 1100, 1101, 1110, and 1111 are invalid




* Unless otherwise specified, we refer to the 8421 BCD code, although several other BCD codes exist, such as 2421, 3321, 4221, 5211, 5311, and 5421.

# Binary Coded Decimal: 8421 BCD

You can think of BCD in terms of column weights in groups of four bits. For an 8-bit BCD number, the column weights are: 80 40 20 10 8 4 2 1.

**Question:** What are the column weights for the BCD number 1000 0011 0101 1001?

**Answer:**

8000 4000 2000 1000 800 400 200 100 80 40 20 10 8 4 2 1

Note that you could add the column weights where there is a 1 to obtain the decimal number. For this case:

$$8000 + 200 + 100 + 40 + 10 + 8 + 1 = 8359_{10}$$

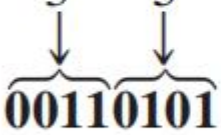
# Binary Coded Decimal: 8421 BCD

Convert each of the following decimal numbers to BCD:

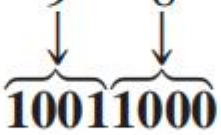
**(a)** 35 **(b)** 98 **(c)** 170 **(d)** 2469

**Solution**

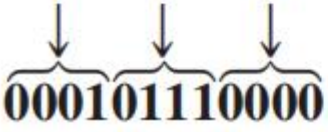
**(a)** 3 5



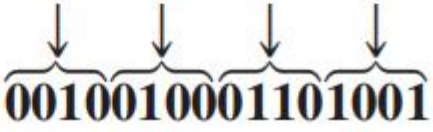

**(b)** 9 8




**(c)** 1 7 0




**(d)** 2 4 6 9



# Binary Coded Decimal: 8421 BCD

Add the following BCD numbers:

**(a)** 1001 + 0100

**(b)** 1001 + 1001

**(c)** 00010110 + 00010101

**(d)** 01100111 + 01010011

**Step 1:** Add the two BCD numbers, using the rules for binary addition in Section 2–4.

**Step 2:** If a 4-bit sum is equal to or less than 9, it is a valid BCD number.

**Step 3:** If a 4-bit sum is greater than 9, or if a carry out of the 4-bit group is generated, it is an invalid result. Add 6 (0110) to the 4-bit sum in order to skip the six invalid states and return the code to 8421. If a carry results when 6 is added, simply add the carry to the next 4-bit group.

# Binary Coded Decimal: 8421 BCD

The decimal number additions are shown for comparison.

**(a)**

| | | | |
|---|---|---|---|
| 1001 | | | 9 |
| + 0100 | | | +4 |
| 1101 | | Invalid BCD number (>9) | 13 |
| + 0110 | | Add 6 | |
| **0001** | **0011** | Valid BCD number | |
| ↓ | ↓ | | |
| 1 | 3 | | |

**(b)**

| | | | |
|---|---|---|---|
| | 1001 | | 9 |
| | + 1001 | | + 9 |
| 1 | 0010 | Invalid because of carry | 18 |
| | + 0110 | Add 6 | |
| **0001** | **1000** | Valid BCD number | |
| ↓ | ↓ | | |
| 1 | 8 | | |

**(c)**

| | | | |
|---|---|---|---|
| 0001 | 0110 | | 16 |
| + 0001 | 0101 | | + 15 |
| 0010 | 1011 | Right group is invalid (>9), left group is valid. | 31 |
| | + 0110 | Add 6 to invalid code. Add carry, 0001, to next group. | |
| **0011** | **0001** | Valid BCD number | |
| ↓ | ↓ | | |
| 3 | 1 | | |

**(d)**

| | | | | |
|---|---|---|---|---|
| | 0110 | 0111 | | 67 |
| | + 0101 | 0011 | | + 53 |
| | 1011 | 1010 | Both groups are invalid (>9) | 120 |
| | + 0110 | + 0110 | Add 6 to both groups | |
| **0001** | **0010** | **0000** | Valid BCD number | |
| ↓ | ↓ | ↓ | | |
| 1 | 2 | 0 | | |

# Alternative BCD Codes

Excess 3 code, 3 is added to the original BCD value and this makes the code ‘reflexive’, that is the top half of the code is a mirror image and the complement of the bottom half, 5211 code is also self-complementing in this way.

Table 1.6.3

| Decimal | 7421 | 5421 | 5211 | 2421 | Excess 3 |
|---|---|---|---|---|---|
| 0 | 0000 | 0000 | 0000 | 0000 | 0011 |
| 1 | 0001 | 0001 | 0010 | 0001 | 0100 |
| 2 | 0010 | 0010 | 0011 | 0010 | 0101 |
| 3 | 0011 | 0011 | 0101 | 0011 | 0110 |
| 4 | 0100 | 0100 | 0111 | 0100 | 0111 |
| 5 | 0101 | 1000 | 1000 | 1011 | 1000 |
| 6 | 0110 | 1001 | 1010 | 1100 | 1001 |
| 7 | 1000 | 1010 | 1100 | 1101 | 1010 |
| 8 | 1001 | 1011 | 1101 | 1110 | 1011 |
| 9 | 1010 | 1100 | 1111 | 1111 | 1100 |

https://learnabout-electronics.org/Digital/dig16.php

# The Gray Code

## Thought exercise: starting from a Leetcode problem:

An **n-bit gray code sequence** is a sequence of $2^n$ integers where:

- Every integer is in the **inclusive** range $[0, 2^n - 1]$,
- The first integer is $0$,
- An integer appears **no more than once** in the sequence,
- The binary representation of every pair of **adjacent** integers differs by **exactly one bit**, and
- The binary representation of the **first** and **last** integers differs by **exactly one bit**.

Given an integer $n$, return *any valid* ***n-bit gray code sequence***.

# The Gray Code

What is unweighted code?

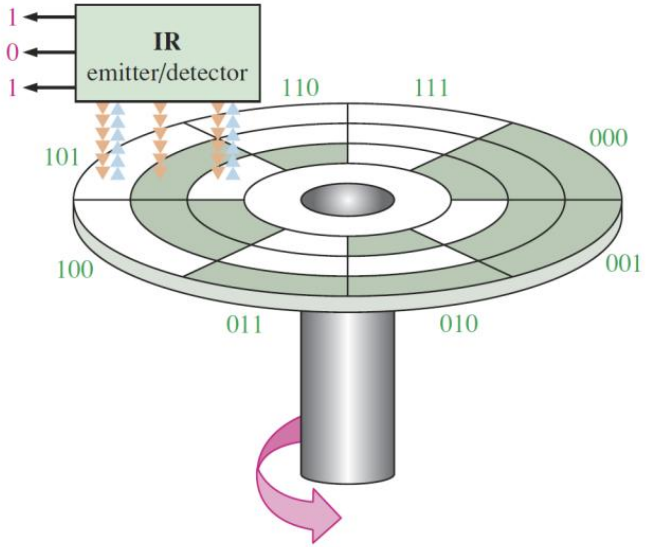
- The Gray code is unweighted.
- An important feature is that it exhibits only a single bit change from one code word to the next in sequence. This property is important in many applications, such as shaft position encoders.

Four-bit Gray code.

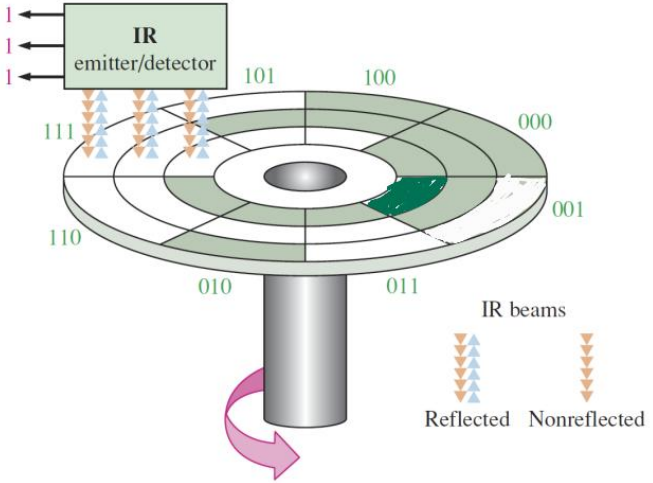
| Decimal | Binary | Gray Code | Decimal | Binary | Gray Code |
|---|---|---|---|---|---|
| 0 | 0000 | 0000 | 8 | 1000 | 1100 |
| 1 | 0001 | 0001 | 9 | 1001 | 1101 |
| 2 | 0010 | 0011 | 10 | 1010 | 1111 |
| 3 | 0011 | 0010 | 11 | 1011 | 1110 |
| 4 | 0100 | 0110 | 12 | 1100 | 1010 |
| 5 | 0101 | 0111 | 13 | 1101 | 1011 |
| 6 | 0110 | 0101 | 14 | 1110 | 1001 |
| 7 | 0111 | 0100 | 15 | 1111 | 1000 |

# The Gray Code

What will happen if Gray code is not used here?

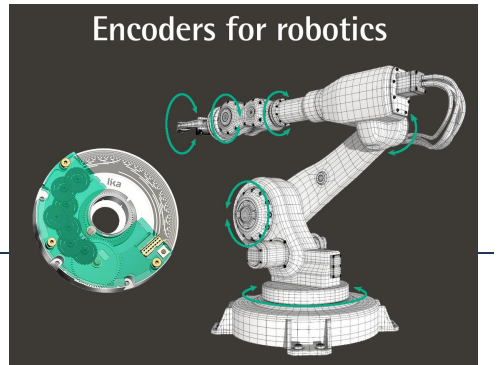



Binary code




Gray code

## Encoder vs Potentiometer?




https://www.ien.eu/uploads/tx_etim/Lika_Electronic_Encoders_for_Robotic_Arms_2_01.jpg

# ASCII

ASCII is a code for alphanumeric characters and control characters. In its original form, ASCII encoded 128 characters and symbols using 7-bits. The first 32 characters are control characters, that are based on obsolete teletype requirements, so these characters are generally assigned to other functions in modern usage.

In 1981, IBM introduced extended ASCII, which is an 8-bit code and increased the character set to 256. Other extended sets (such as Unicode) have been introduced to handle characters in languages other than English.

**TABLE 2–7**

American Standard Code for Information Interchange (ASCII).

| Control Characters | | | | Graphic Symbols | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Name | Dec | Binary | Hex | Symbol | Dec | Binary | Hex | Symbol | Dec | Binary | Hex | Symbol | Dec | Binary | Hex |
| NUL | 0 | 0000000 | 00 | space | 32 | 0100000 | 20 | @ | 64 | 1000000 | 40 | ′ | 96 | 1100000 | 60 |
| SOH | 1 | 0000001 | 01 | ! | 33 | 0100001 | 21 | A | 65 | 1000001 | 41 | a | 97 | 1100001 | 61 |
| STX | 2 | 0000010 | 02 | ” | 34 | 0100010 | 22 | B | 66 | 1000010 | 42 | b | 98 | 1100010 | 62 |
| ETX | 3 | 0000011 | 03 | # | 35 | 0100011 | 23 | C | 67 | 1000011 | 43 | c | 99 | 1100011 | 63 |
| EOT | 4 | 0000100 | 04 | $ | 36 | 0100100 | 24 | D | 68 | 1000100 | 44 | d | 100 | 1100100 | 64 |
| ENQ | 5 | 0000101 | 05 | % | 37 | 0100101 | 25 | E | 69 | 1000101 | 45 | e | 101 | 1100101 | 65 |
| ACK | 6 | 0000110 | 06 | & | 38 | 0100110 | 26 | F | 70 | 1000110 | 46 | f | 102 | 1100110 | 66 |
| BEL | 7 | 0000111 | 07 | ’ | 39 | 0100111 | 27 | G | 71 | 1000111 | 47 | g | 103 | 1100111 | 67 |
| BS | 8 | 0001000 | 08 | ( | 40 | 0101000 | 28 | H | 72 | 1001000 | 48 | h | 104 | 1101000 | 68 |
| HT | 9 | 0001001 | 09 | ) | 41 | 0101001 | 29 | I | 73 | 1001001 | 49 | i | 105 | 1101001 | 69 |
| LF | 10 | 0001010 | 0A | * | 42 | 0101010 | 2A | J | 74 | 1001010 | 4A | j | 106 | 1101010 | 6A |
| VT | 11 | 0001011 | 0B | + | 43 | 0101011 | 2B | K | 75 | 1001011 | 4B | k | 107 | 1101011 | 6B |
| FF | 12 | 0001100 | 0C | , | 44 | 0101100 | 2C | L | 76 | 1001100 | 4C | l | 108 | 1101100 | 6C |
| CR | 13 | 0001101 | 0D | − | 45 | 0101101 | 2D | M | 77 | 1001101 | 4D | m | 109 | 1101101 | 6D |
| SO | 14 | 0001110 | 0E | . | 46 | 0101110 | 2E | N | 78 | 1001110 | 4E | n | 110 | 1101110 | 6E |
| SI | 15 | 0001111 | 0F | / | 47 | 0101111 | 2F | O | 79 | 1001111 | 4F | o | 111 | 1101111 | 6F |
| DLE | 16 | 0010000 | 10 | 0 | 48 | 0110000 | 30 | P | 80 | 1010000 | 50 | p | 112 | 1110000 | 70 |
| DC1 | 17 | 0010001 | 11 | 1 | 49 | 0110001 | 31 | Q | 81 | 1010001 | 51 | q | 113 | 1110001 | 71 |
| DC2 | 18 | 0010010 | 12 | 2 | 50 | 0110010 | 32 | R | 82 | 1010010 | 52 | r | 114 | 1110010 | 72 |
| DC3 | 19 | 0010011 | 13 | 3 | 51 | 0110011 | 33 | S | 83 | 1010011 | 53 | s | 115 | 1110011 | 73 |
| DC4 | 20 | 0010100 | 14 | 4 | 52 | 0110100 | 34 | T | 84 | 1010100 | 54 | t | 116 | 1110100 | 74 |
| NAK | 21 | 0010101 | 15 | 5 | 53 | 0110101 | 35 | U | 85 | 1010101 | 55 | u | 117 | 1110101 | 75 |
| SYN | 22 | 0010110 | 16 | 6 | 54 | 0110110 | 36 | V | 86 | 1010110 | 56 | v | 118 | 1110110 | 76 |
| ETB | 23 | 0010111 | 17 | 7 | 55 | 0110111 | 37 | W | 87 | 1010111 | 57 | w | 119 | 1110111 | 77 |
| CAN | 24 | 0011000 | 18 | 8 | 56 | 0111000 | 38 | X | 88 | 1011000 | 58 | x | 120 | 1111000 | 78 |
| EM | 25 | 0011001 | 19 | 9 | 57 | 0111001 | 39 | Y | 89 | 1011001 | 59 | y | 121 | 1111001 | 79 |
| SUB | 26 | 0011010 | 1A | : | 58 | 0111010 | 3A | Z | 90 | 1011010 | 5A | z | 122 | 1111010 | 7A |
| ESC | 27 | 0011011 | 1B | ; | 59 | 0111011 | 3B | [ | 91 | 1011011 | 5B | { | 123 | 1111011 | 7B |
| FS | 28 | 0011100 | 1C | < | 60 | 0111100 | 3C | \ | 92 | 1011100 | 5C | \| | 124 | 1111100 | 7C |
| GS | 29 | 0011101 | 1D | = | 61 | 0111101 | 3D | ] | 93 | 1011101 | 5D | } | 125 | 1111101 | 7D |
| RS | 30 | 0011110 | 1E | > | 62 | 0111110 | 3E | ^ | 94 | 1011110 | 5E | ~ | 126 | 1111110 | 7E |
| US | 31 | 0011111 | 1F | ? | 63 | 0111111 | 3F | _ | 95 | 1011111 | 5F | Del | 127 | 1111111 | 7F |


Copyright © 2015 by Pearson Education, Inc.
All Rights Reserved
PEARSON

# ASCII exercises

- Convert ASCII Values to Characters
- Convert Lowercase to Uppercase
- Check if Character is Alphabet
- Remove All Digits from String
- How many characters between ‘a’ and ‘m’?

**TABLE 2–7**

American Standard Code for Information Interchange (ASCII).

| Control Characters | | | | Graphic Symbols | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Name | Dec | Binary | Hex | Symbol | Dec | Binary | Hex | Symbol | Dec | Binary | Hex | Symbol | Dec | Binary | Hex |
| NUL | 0 | 0000000 | 00 | space | 32 | 0100000 | 20 | @ | 64 | 1000000 | 40 | ′ | 96 | 1100000 | 60 |
| SOH | 1 | 0000001 | 01 | ! | 33 | 0100001 | 21 | A | 65 | 1000001 | 41 | a | 97 | 1100001 | 61 |
| STX | 2 | 0000010 | 02 | ” | 34 | 0100010 | 22 | B | 66 | 1000010 | 42 | b | 98 | 1100010 | 62 |
| ETX | 3 | 0000011 | 03 | # | 35 | 0100011 | 23 | C | 67 | 1000011 | 43 | c | 99 | 1100011 | 63 |
| EOT | 4 | 0000100 | 04 | $ | 36 | 0100100 | 24 | D | 68 | 1000100 | 44 | d | 100 | 1100100 | 64 |
| ENQ | 5 | 0000101 | 05 | % | 37 | 0100101 | 25 | E | 69 | 1000101 | 45 | e | 101 | 1100101 | 65 |
| ACK | 6 | 0000110 | 06 | & | 38 | 0100110 | 26 | F | 70 | 1000110 | 46 | f | 102 | 1100110 | 66 |
| BEL | 7 | 0000111 | 07 | ’ | 39 | 0100111 | 27 | G | 71 | 1000111 | 47 | g | 103 | 1100111 | 67 |
| BS | 8 | 0001000 | 08 | ( | 40 | 0101000 | 28 | H | 72 | 1001000 | 48 | h | 104 | 1101000 | 68 |
| HT | 9 | 0001001 | 09 | ) | 41 | 0101001 | 29 | I | 73 | 1001001 | 49 | i | 105 | 1101001 | 69 |
| LF | 10 | 0001010 | 0A | * | 42 | 0101010 | 2A | J | 74 | 1001010 | 4A | j | 106 | 1101010 | 6A |
| VT | 11 | 0001011 | 0B | + | 43 | 0101011 | 2B | K | 75 | 1001011 | 4B | k | 107 | 1101011 | 6B |
| FF | 12 | 0001100 | 0C | , | 44 | 0101100 | 2C | L | 76 | 1001100 | 4C | l | 108 | 1101100 | 6C |
| CR | 13 | 0001101 | 0D | − | 45 | 0101101 | 2D | M | 77 | 1001101 | 4D | m | 109 | 1101101 | 6D |
| SO | 14 | 0001110 | 0E | . | 46 | 0101110 | 2E | N | 78 | 1001110 | 4E | n | 110 | 1101110 | 6E |
| SI | 15 | 0001111 | 0F | / | 47 | 0101111 | 2F | O | 79 | 1001111 | 4F | o | 111 | 1101111 | 6F |
| DLE | 16 | 0010000 | 10 | 0 | 48 | 0110000 | 30 | P | 80 | 1010000 | 50 | p | 112 | 1110000 | 70 |
| DC1 | 17 | 0010001 | 11 | 1 | 49 | 0110001 | 31 | Q | 81 | 1010001 | 51 | q | 113 | 1110001 | 71 |
| DC2 | 18 | 0010010 | 12 | 2 | 50 | 0110010 | 32 | R | 82 | 1010010 | 52 | r | 114 | 1110010 | 72 |
| DC3 | 19 | 0010011 | 13 | 3 | 51 | 0110011 | 33 | S | 83 | 1010011 | 53 | s | 115 | 1110011 | 73 |
| DC4 | 20 | 0010100 | 14 | 4 | 52 | 0110100 | 34 | T | 84 | 1010100 | 54 | t | 116 | 1110100 | 74 |
| NAK | 21 | 0010101 | 15 | 5 | 53 | 0110101 | 35 | U | 85 | 1010101 | 55 | u | 117 | 1110101 | 75 |
| SYN | 22 | 0010110 | 16 | 6 | 54 | 0110110 | 36 | V | 86 | 1010110 | 56 | v | 118 | 1110110 | 76 |
| ETB | 23 | 0010111 | 17 | 7 | 55 | 0110111 | 37 | W | 87 | 1010111 | 57 | w | 119 | 1110111 | 77 |
| CAN | 24 | 0011000 | 18 | 8 | 56 | 0111000 | 38 | X | 88 | 1011000 | 58 | x | 120 | 1111000 | 78 |
| EM | 25 | 0011001 | 19 | 9 | 57 | 0111001 | 39 | Y | 89 | 1011001 | 59 | y | 121 | 1111001 | 79 |
| SUB | 26 | 0011010 | 1A | : | 58 | 0111010 | 3A | Z | 90 | 1011010 | 5A | z | 122 | 1111010 | 7A |
| ESC | 27 | 0011011 | 1B | ; | 59 | 0111011 | 3B | [ | 91 | 1011011 | 5B | { | 123 | 1111011 | 7B |
| FS | 28 | 0011100 | 1C | < | 60 | 0111100 | 3C | \ | 92 | 1011100 | 5C | l | 124 | 1111100 | 7C |
| GS | 29 | 0011101 | 1D | = | 61 | 0111101 | 3D | ] | 93 | 1011101 | 5D | } | 125 | 1111101 | 7D |
| RS | 30 | 0011110 | 1E | > | 62 | 0111110 | 3E | ^ | 94 | 1011110 | 5E | ~ | 126 | 1111110 | 7E |
| US | 31 | 0011111 | 1F | ? | 63 | 0111111 | 3F | _ | 95 | 1011111 | 5F | Del | 127 | 1111111 | 7F |

# ASCII exercises

## Python: ASCII manipulation

**Source Code**

```
# Program to find the ASCII value of the given character

c = 'p'
print("The ASCII value of '" + c + "' is", ord(c))
```

Run Code »

**Output**

```
The ASCII value of 'p' is 112
```

**Your turn:** Modify the code above to get characters from their corresponding ASCII values using the **chr()** function as shown below.

```
>>> chr(65)
'A'
>>> chr(120)
'x'
>>> chr(ord('S') + 1)
'T'
```

Here, `ord()` and `chr()` are built-in functions.

https://www.programiz.com/python-programming/examples/ascii-character

# Error Detection: Parity Method

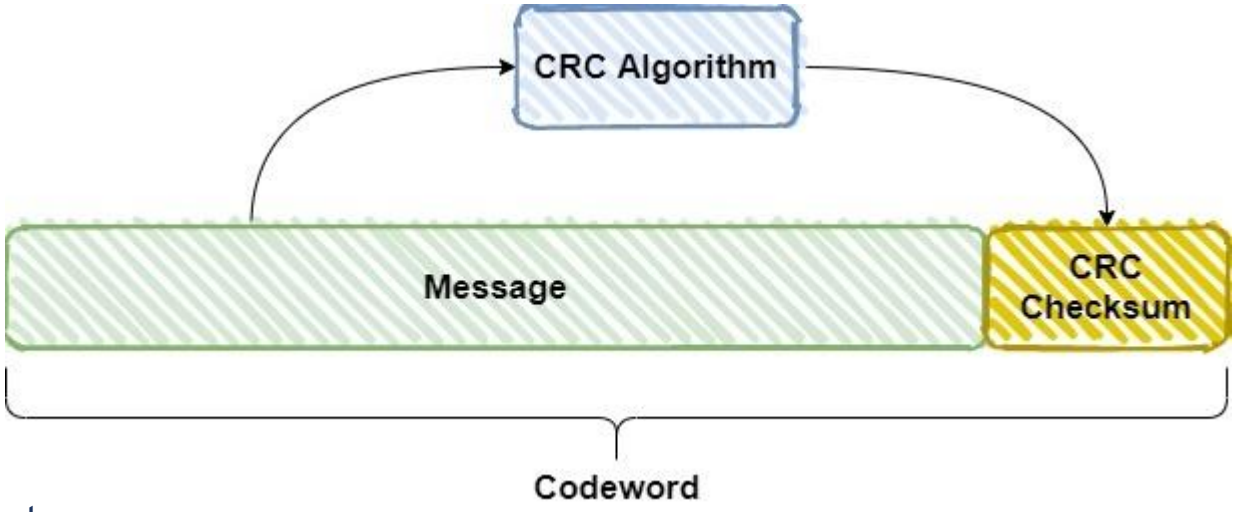
- Any group of bits contain either an even or an odd number of 1s. An even (odd) parity bit makes the total number of 1s even (odd).
- A parity bit provides for the detection of a single bit error but cannot check for two errors

The BCD code with parity bits.

| Even Parity | | Odd Parity | |
|---|---|---|---|
| *P* | BCD | *P* | BCD |
| 0 | 0000 | 1 | 0000 |
| 1 | 0001 | 0 | 0001 |
| 1 | 0010 | 0 | 0010 |
| 0 | 0011 | 1 | 0011 |
| 1 | 0100 | 0 | 0100 |
| 0 | 0101 | 1 | 0101 |
| 0 | 0110 | 1 | 0110 |
| 1 | 0111 | 0 | 0111 |
| 1 | 1000 | 0 | 1000 |
| 0 | 1001 | 1 | 1001 |

# Error Detection: Cyclic Redundancy Check

The cyclic redundancy check (CRC) is an error detection method that can detect multiple errors in larger blocks of data. At the sending end, a checksum is appended to a block of data. At the receiving end, the check sum is generated and compared to the sent checksum. If the check sums are the same, no error is detected.




https://www.eet-china.com/mp/a59789.html

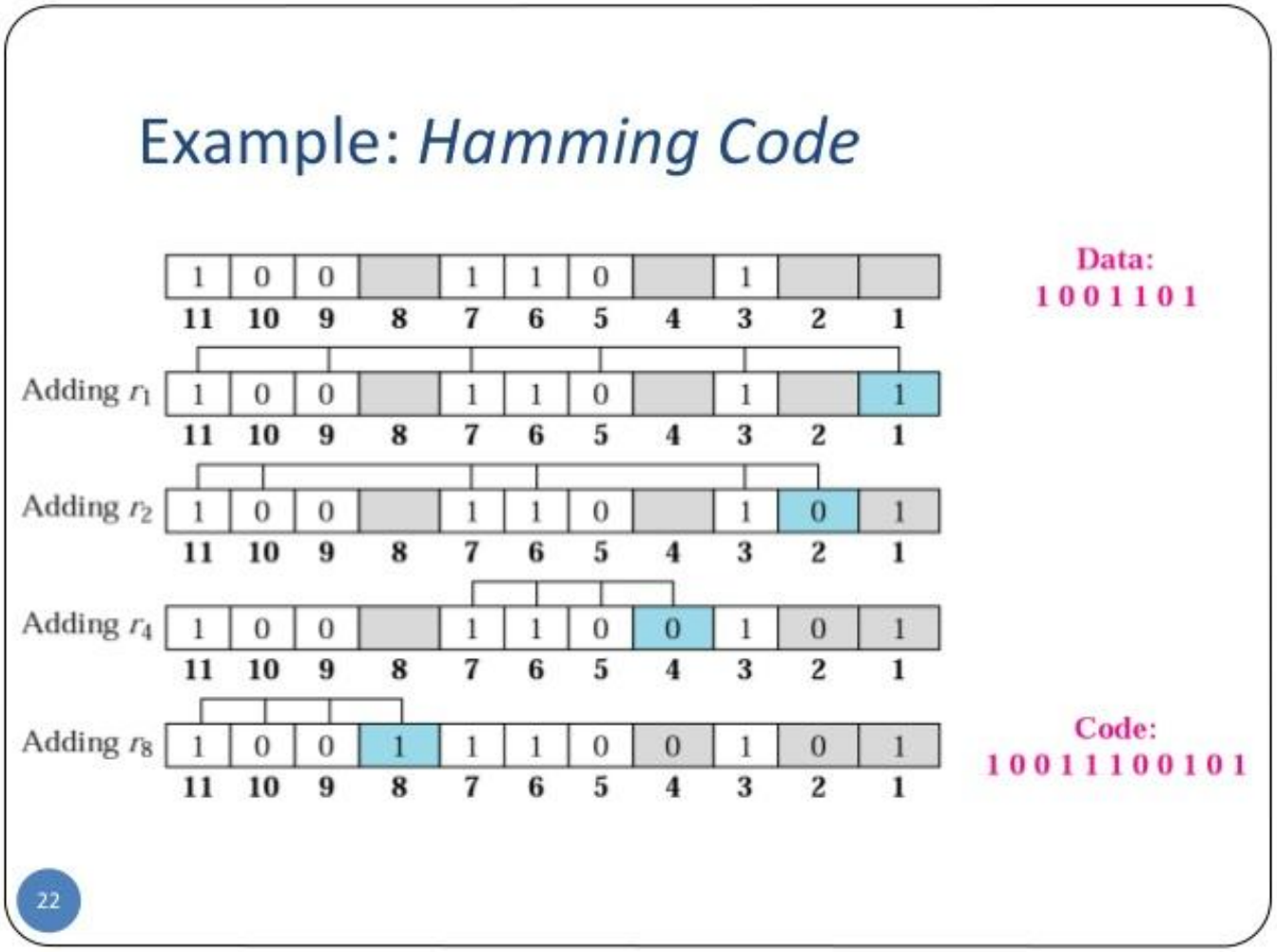
# Error Detection: Hamming Code

- Hamming code is an error-correcting code used to ensure data accuracy during transmission or storage.
- Hamming code detects and corrects the errors that can occur when the data is moved or stored from the sender to the receiver. This simple and effective method helps improve the reliability of communication systems and digital storage.
- It adds extra bits to the original data, allowing the system to detect and correct single-bit errors. It is a technique developed by Richard Hamming in the 1950s.



https://www.geeksforgeeks.org/hamming-code-in-computer-network/

# Error Detection: Hamming Code






https://www.thecrazyprogrammer.com/2017/03/hamming-code-c.html